

MASTER PROMPT — AGRICHAIN

=====

PROJECT NAME:

AgriChain — Trusted Digital Passport for Agricultural Cold-Chain Traceability

PROJECT TYPE:

Full-stack IoT + Blockchain + Web Application Prototype

CORE CONCEPT:

Build a production-style official web portal for agricultural export traceability.

The platform connects simulated IoT sensor devices, MQTT-style telemetry, secure data processing, cryptographic verification, Hyperledger Fabric blockchain records, shipment tracking, alerts, chain-of-custody management, and QR-based digital product passports.

IMPORTANT:

For this version, DO NOT build the physical ESP32/IoT hardware firmware or actual hardware automation.

Instead, create a fully working IoT SIMULATION ENGINE INSIDE THE WEB APPLICATION.

The simulation must behave as if real IoT devices are connected.

The architecture MUST be designed so that the simulated IoT layer can later be replaced by real ESP32 + MQTT devices without redesigning the application.

The project must feel like a serious commercial SaaS/enterprise platform, NOT a college-project-looking website.

=====

1. PRODUCT VISION

=====

Build a platform that gives every agricultural export shipment a trusted digital identity.

The complete journey:

Farm



Warehouse



Transport



Port



Exporter



Importer

During transportation, a sensor device continuously monitors:

- Temperature
- Humidity
- Ethylene gas
- GPS/location
- Battery level
- Connectivity
- Device tampering status

The application must simulate these sensor values in real time.

The system must:

1. Create shipments
2. Assign IoT devices
3. Simulate live sensor readings
4. Store telemetry
5. Detect abnormal conditions
6. Generate alerts
7. Simulate connectivity loss
8. Continue collecting data while offline
9. Automatically synchronize data after reconnection
10. Generate cryptographic hashes for sensor batches
11. Record important events on Hyperledger Fabric
12. Maintain an immutable chain of custody
13. Detect data tampering
14. Generate QR-based digital passports
15. Allow authorized users to verify shipment authenticity
16. Provide analytics and reports

=====

2. DESIGN PHILOSOPHY

=====

The website MUST feel:

- Human-designed
- Professional
- Trustworthy
- Modern
- Easy to understand
- Enterprise-grade
- Official
- Clean

- Responsive
- Fast
- Accessible

DO NOT make it look like:

- A generic AI-generated dashboard
- A template with excessive gradients
- A gaming UI
- A futuristic cyberpunk blockchain website
- An over-animated landing page
- A university mini-project

The visual identity should communicate:

TRUST + AGRICULTURE + TECHNOLOGY + TRACEABILITY

Use a refined visual language inspired by modern enterprise products.

Use:

- Clean whitespace
- Strong typography
- Subtle borders
- Soft shadows
- Rounded but professional cards
- Clear hierarchy
- Consistent spacing
- Minimal animations
- Meaningful icons
- Professional charts

- Clear status indicators

Avoid excessive:

- Glassmorphism
- Neon colors
- Huge gradients
- Floating objects everywhere
- Excessive rounded cards
- Unnecessary animations

=====

3. BRANDING

=====

Product name:

AGRICHAIN

Tagline:

"Trust Every Journey."

Alternative supporting statement:

"Transparent agricultural traceability from farm to destination."

Create a professional logo using:

- Leaf
- Chain/link

- Digital node

Do NOT use copyrighted logos.

Brand personality:

Reliable

Transparent

Secure

Sustainable

Practical

Suggested color direction:

Primary:

Deep agricultural green

Secondary:

Dark charcoal / navy

Accent:

Warm amber

Success:

Natural green

Warning:

Amber

Danger:

Red

Background:

Off-white / very light gray

Do not overuse colors.

=====

4. USER TYPES

=====

Create role-based access.

Roles:

1. ADMIN
2. FARMER
3. EXPORTER
4. LOGISTICS PROVIDER
5. IMPORTER
6. AUDITOR

Each role should see appropriate information.

ADMIN:

- All shipments
- Users
- Devices
- Blockchain
- Alerts
- Analytics
- System settings

FARMER:

- Create shipment
- View own shipments
- Assign sensor
- Monitor shipment
- Generate QR passport

EXPORTER:

- Monitor shipments
- Verify chain of custody
- View environmental conditions
- Download reports
- Verify blockchain integrity

LOGISTICS:

- View assigned shipments
- Monitor live sensor telemetry
- Update custody
- Respond to alerts

IMPORTER:

- Verify incoming shipment
- Scan QR
- View digital passport
- Verify blockchain integrity

AUDITOR:

- Read-only access
- Blockchain records
- Chain of custody

- Sensor history
- Integrity verification
- Export audit report

=====

5. PUBLIC LANDING PAGE

=====

Create a professional public landing page.

Header:

AGRICHAIN logo

Navigation:

Home

How It Works

Traceability

Technology

For Businesses

About

Buttons:

Login

Get Started

Hero section:

Headline:

"Trust Every Journey."

Subheadline:

"Secure, affordable and tamper-evident traceability for agricultural supply chains — from farm to final destination."

Primary CTA:

"Start Tracking"

Secondary CTA:

"Explore Traceability"

Hero visual:

Create a realistic supply-chain visualization:

FARM



WAREHOUSE



TRANSPORT



PORT



EXPORT



IMPORTER

Show subtle animated data flowing through the chain.

Do NOT make the animation distracting.

Landing page sections:

1. Problem
2. Solution
3. How It Works
4. IoT Monitoring
5. Blockchain Integrity
6. Digital Product Passport
7. Benefits for SMEs
8. Call to Action
9. Footer

Problem section:

Explain:

Agricultural exporters face:

- Supply-chain blind spots
- Connectivity loss
- Temperature excursions
- Data manipulation risks
- Expensive enterprise tracking systems

Solution:

AgriChain provides:

- Real-time environmental monitoring
- Offline-first data collection
- Secure synchronization
- Blockchain-backed traceability
- QR digital passports
- SME-friendly architecture

=====

6. AUTHENTICATION

=====

Create a professional authentication system.

Pages:

/login

/register

/forgot-password

Login screen:

AGRICHAIN

"Welcome back"

Email

Password

Remember me

Forgot password?

Login button

"Don't have an account? Create one"

Provide DEMO LOGIN accounts.

Example:

Admin:

admin@agrchain.demo

password: Admin@123

Farmer:

farmer@agrchain.demo

password: Farmer@123

Exporter:

exporter@agrchain.demo

password: Exporter@123

Logistics:

logistics@agrchain.demo

password: Logistics@123

Auditor:

auditor@agrchain.demo

password: Auditor@123

The application must work immediately using these demo accounts.

Do not require real email verification for the prototype.

=====

7. APPLICATION SHELL

=====

After login, use a professional application layout.

LEFT SIDEBAR:

Dashboard

Shipments

Live Monitoring

IoT Devices

Alerts

Blockchain

Chain of Custody

Digital Passports

Analytics

Reports

Users

Settings

Top bar:

Search

Notifications

Connection status

User profile

Role

Sidebar must collapse on smaller screens.

Mobile navigation must remain usable.

=====

8. DASHBOARD

=====

Dashboard title:

"Supply Chain Overview"

Top KPI cards:

Active Shipments

Safe Shipments

Attention Required

Critical Alerts

Additional:

Connected Devices

Offline Devices

Blockchain Verified

Data Synced

Example:

42 Active Shipments

35 Safe

5 Attention

2 Critical

Do not hard-code these forever.

Use dynamic application state.

Dashboard sections:

A. Shipment overview

B. Live sensor activity

C. Recent alerts

D. Recent blockchain transactions

E. Supply chain map

F. Device health

G. Sync status

=====

9. IOT SIMULATION ENGINE

=====

THIS IS VERY IMPORTANT.

Since real hardware is not being implemented yet, create a fully functional IoT SIMULATION ENGINE.

The simulation must act like actual IoT devices.

Create device IDs:

AG-SENSOR-001

AG-SENSOR-002

AG-SENSOR-003

AG-SENSOR-004

AG-SENSOR-005

Each device should have:

deviceId

shipmentId

temperature

humidity

ethylene

latitude

longitude

battery

signalStrength

connectivity

tamperStatus

lastSeen

firmwareVersion

The simulator should continuously generate readings.

Default interval:

Every 3–5 seconds for demo purposes.

Example:

Temperature:

5–12°C

Humidity:

55–85%

Ethylene:

0.5–4 ppm

Battery:

0–100%

Signal:

-50 to -110 dBm

GPS:

Realistic movement along a predefined route.

=====

10. SENSOR SIMULATION CONTROLS

=====

Create a dedicated page:

/iot-devices

Show:

Device cards/table.

Each device:

Device ID

Assigned Shipment

Status

Battery

Signal

Temperature

Humidity

Ethylene

Last Seen

Provide:

START SIMULATION

PAUSE

RESET

Also create DEMO SCENARIO controls.

Scenario buttons:

NORMAL

TEMPERATURE SPIKE

HIGH HUMIDITY

ETHYLENE SPIKE

NETWORK OUTAGE

DEVICE TAMPER

BATTERY LOW

FULL RECOVERY

These scenarios must actually change the simulated data.

=====

11. LIVE MONITORING

=====

Page:

/monitoring

Header:

"Live Shipment Monitoring"

Shipment selector.

Show:

Temperature chart

Humidity chart

Ethylene chart

Battery chart

Use real-time chart updates.

Example:

Temperature:

7.2°C

Humidity:

68%

Ethylene:

1.4 ppm

Battery:

87%

Connectivity:

ONLINE

Use clear status badges.

=====

12. REALISTIC SENSOR BEHAVIOR

=====

Do NOT generate completely random numbers.

Use realistic gradual changes.

For example:

Temperature:

7.2

7.3

7.4

7.5

7.4

7.6

Instead of:

2

11

4

19

3

Implement controlled random drift.

When a scenario is activated:

TEMPERATURE SPIKE:

7°C

7.5°C

8°C

10°C

12°C

14°C

Then trigger an alert.

=====

13. OFFLINE MODE SIMULATION

=====

This is one of the most important features.

Create a "Network Offline" simulation.

When activated:

Connectivity:

OFFLINE

The system MUST:

- Continue generating sensor readings
- Store them locally
- Mark them as pending synchronization
- Show offline banner
- Display number of pending records

Example:

NETWORK OFFLINE

"Sensor data is being securely stored locally."

Pending records:

When network is restored:

"Connection restored"

Then automatically:

1. Read pending data
2. Validate records
3. Generate cryptographic hash
4. Synchronize
5. Record blockchain transaction
6. Mark records as verified
7. Update UI

Show a synchronization animation:

Syncing 24 records...

24 / 24

Synchronization complete ✓

=====

14. SENSOR DATA STORAGE

=====

Use a clean data model.

Telemetry:

{

```
id,  
deviceId,  
shipmentId,  
timestamp,  
temperature,  
humidity,  
ethylene,  
latitude,  
longitude,  
battery,  
signalStrength,  
connectivity,  
encrypted,  
synced,  
blockchainVerified,  
hash  
}
```

Use localStorage / IndexedDB for frontend simulation where appropriate.

If backend exists, persist important records in the backend database.

=====

15. CRYPTOGRAPHIC DATA INTEGRITY

=====

Implement real hashing.

For each synchronized telemetry batch:

Create canonical JSON.

Generate:

SHA-256 hash.

Example:

sensor data

↓

canonical JSON

↓

SHA-256

↓

dataHash

Store:

dataHash

timestamp

shipmentId

deviceId

The UI should display:

Data Integrity:

VERIFIED ✓

=====

16. TAMPERING DEMONSTRATION

=====

Create a controlled demo feature:

"Simulate Data Tampering"

When activated:

Take an existing telemetry record.

Change:

Temperature:

7.4°C

to:

17.4°C

WITHOUT changing blockchain hash.

Run integrity verification.

Expected result:

BLOCKCHAIN HASH

8f72a...

CURRENT DATA HASH

92ab1...

RESULT:

DATA INTEGRITY FAILED

Show a prominent alert:

"Tampering detected. The current sensor data does not match the blockchain-anchored record."

This must actually work.

=====

17. HYPERLEDGER FABRIC INTEGRATION

=====

Use Hyperledger Fabric as the blockchain layer.

The architecture should include:

Fabric Network

Fabric CA

Peers

Orderer

Channel

Chaincode

Create a backend blockchain service abstraction.

Example:

```
blockchainService.createShipment()
```

```
blockchainService.recordSensorBatch()
```

```
blockchainService.transferCustody()
```

```
blockchainService.recordAlert()
```

blockchainService.verifyIntegrity()

IMPORTANT:

Do not tightly couple frontend directly to Fabric.

Architecture:

React



Backend API



Blockchain Service



Hyperledger Fabric

=====

18. DEVELOPMENT FALLBACK

=====

If a live Hyperledger Fabric network is unavailable in the current environment:

Create a clearly separated:

BLOCKCHAIN MOCK ADAPTER

The application should continue functioning.

Structure:

BlockchainProvider

└─ HyperledgerFabricProvider
└─ MockBlockchainProvider

The default development provider can be MockBlockchainProvider.

BUT:

The UI must behave exactly as if blockchain transactions are occurring.

Every mock transaction must have:

transactionId

blockNumber

timestamp

hash

status

Example:

Transaction:

TX-8F92A7

Block:

#1042

Status:

CONFIRMED

The UI should clearly label development/mock blockchain data internally where appropriate.

Do NOT fake a real Hyperledger network and claim it is connected.

=====

19. SMART CONTRACT / CHAINCODE MODEL

=====

Create chaincode design for:

createShipment
registerDevice
recordSensorBatch
transferCustody
recordAlert
recordTamperEvent
updateShipmentStatus
verifyIntegrity
closeShipment

Shipment asset:

```
{  
  shipmentId,  
  product,  
  origin,  
  destination,  
  producer,  
  currentOwner,  
  sensorId,  
  status,  
  createdAt,  
  updatedAt,  
  lastSensorHash
```

}

=====

20. BLOCKCHAIN EXPLORER

=====

Page:

/blockchain

Show:

Total Transactions

Latest Block

Verified Records

Integrity Failures

Transaction table:

Transaction ID

Type

Shipment

Timestamp

Block

Status

Example:

TX-09283

SENSOR_BATCH

MANGO-001

10:32 AM

#1042

CONFIRMED

Click transaction.

Show:

Transaction details

Block number

Timestamp

Hash

Previous hash

Shipment

Device

Event type

=====

21. CHAIN OF CUSTODY

=====

Page:

/chain-of-custody/:shipmentId

Visual timeline:

FARM

✓ Created

WAREHOUSE

✓ Received

TRANSPORT

✓ Dispatched

PORT

✓ Arrived

EXPORT

✓ Cleared

IMPORTER

Current

Each step:

Organization

Person/role

Timestamp

Location

Transaction ID

Blockchain status

Use a beautiful vertical timeline.

=====

22. SHIPMENT MANAGEMENT

=====

Page:

/shipments

Create Shipment button.

Shipment fields:

Shipment ID

Product

Product category

Quantity

Origin

Destination

Producer

Expected delivery

Sensor device

Transporter

Shipment status:

CREATED

AT_FARM

IN_WAREHOUSE

IN_TRANSIT

AT_PORT

EXPORTED

DELIVERED

ALERT

CLOSED

Create realistic sample shipments.

Example products:

Mango

Banana

Tomato

Seafood

Pharmaceutical produce

=====

23. SHIPMENT DETAIL PAGE

=====

Page:

/shipments/:id

Show:

Shipment header

MANGO-001

Status:

IN TRANSIT

Origin:

Erode, Tamil Nadu

Destination:

Dubai, UAE

Then:

Live environmental conditions

Temperature

Humidity

Ethylene

Battery

Map

Timeline

Sensor graph

Alerts

Blockchain verification

Chain of custody

Digital passport

=====

24. DIGITAL PRODUCT PASSPORT

=====

Every shipment must have a QR code.

Generate QR code from:

shipmentId

Create:

/passport/:shipmentId

Public-friendly page.

It should show:

AGRICHAIN

DIGITAL PRODUCT PASSPORT

Product

Origin

Destination

Harvest date

Shipment ID

Current status

Environmental summary:

Temperature:

Within expected range ✓

Humidity:

Within expected range ✓

Ethylene:

Within expected range ✓

Traceability:

4 custody events

Blockchain:

VERIFIED ✓

Data integrity:

VERIFIED ✓

Provide:

"View Full Traceability"

Do not expose sensitive user information.

=====

25. QR SCANNER

=====

Create:

/verify

Allow users to scan QR codes using device camera where browser permissions allow.

Also provide manual shipment ID input as fallback.

After scanning:

Load digital passport.

Show:

VERIFIED

or

WARNING

or

INTEGRITY FAILURE

=====

26. ALERT ENGINE

=====

Create a rule-based alert engine.

Temperature:

Normal:

2–8°C

Warning:

8–10°C

Critical:

>10°C

Humidity:

Warning:

>80%

Ethylene:

Warning:

>3 ppm

Battery:

Warning:

<20%

Critical:

<10%

Connectivity:

Offline > configured threshold

Tamper:

Immediately critical.

Alerts should be generated automatically.

=====

27. ALERT CENTER

=====

Page:

/alerts

Filters:

All

Critical

Warning

Resolved

Each alert:

Type

Shipment

Device

Value

Expected range

Timestamp

Status

Actions:

Acknowledge

Resolve

View Shipment

=====

28. ANALYTICS

=====

Page:

/analytics

Charts:

Temperature compliance

Humidity compliance

Ethylene trends

Shipment duration

Alert frequency

Device uptime

Offline duration

Blockchain transactions

Shipment success rate

Filters:

Date

Product

Origin

Destination

Shipment

Use professional charts.

=====

29. REPORT GENERATION

=====

Create a report page.

Allow:

Generate Shipment Report

Report should contain:

Shipment information

Environmental summary

Alert history

Chain of custody

Blockchain verification

Sensor integrity

Final status

Allow:

Print

Download

If PDF generation is practical, implement it.

=====

30. USER EXPERIENCE

=====

The application **MUST** be extremely easy to use.

Rules:

- Never overload the user
- Use plain language

- Use tooltips
- Use meaningful empty states
- Use confirmation dialogs
- Use loading states
- Use success messages
- Use error messages
- Use skeleton loading
- Use responsive tables
- Use pagination where appropriate
- Use search and filters

Example:

Instead of:

"MQTT_CONNECTION_EXCEPTION"

show:

"Device connection interrupted. Sensor data is being stored locally."

Instead of:

"Blockchain transaction failed"

show:

"We couldn't verify this record. Please try again."

=====

31. NOTIFICATIONS

=====

Create notification center.

Examples:

"Shipment MANGO-001 exceeded the temperature threshold."

"Network restored. 24 sensor records synchronized."

"Blockchain verification successful."

"Device AG-SENSOR-003 battery is below 20%."

=====

32. SETTINGS

=====

Page:

/settings

Sections:

Profile

Organization

Notification preferences

Temperature thresholds

Humidity thresholds

Ethylene thresholds

Security

Blockchain

IoT simulation

Appearance

Allow demo threshold editing.

=====

33. DATABASE

=====

Use a clean schema.

Collections/tables:

users

organizations

shipments

devices

telemetry

alerts

custodyEvents

blockchainTransactions

notifications

Create relationships logically.

=====

34. API DESIGN

=====

Create clean REST APIs.

Examples:

POST /api/auth/login

GET /api/shipments

POST /api/shipments

GET /api/shipments/:id

POST /api/shipments/:id/custody

GET /api/shipments/:id/telemetry

POST /api/telemetry

GET /api/devices

POST /api/devices

POST /api/simulation/start

POST /api/simulation/pause

POST /api/simulation/scenario

GET /api/alerts

POST /api/alerts/:id/resolve

GET /api/blockchain/transactions

GET /api/blockchain/verify/:shipmentId

=====

35. REAL-TIME ARCHITECTURE

=====

For simulated sensor data:

IoT Simulator

↓

Event system

↓

Backend

↓

WebSocket / Socket.IO

↓

React Dashboard

The dashboard must update without refreshing the browser.

If Socket.IO is unavailable, implement a clean polling fallback.

=====

36. DEMO DATA

=====

Create realistic demo data.

At least:

10 shipments

5 devices

Multiple custody events

50+ telemetry records

10 alerts

20 blockchain transactions

Create different shipment states:

Safe

Warning

Critical

Offline

Delivered

Awaiting export

=====

37. DEMO MODE

=====

Create a special:

"Demo Mode"

button.

When enabled, provide:

Start Normal Journey

Temperature Breach

Network Dropout

Tampering Attack

Recovery

These should execute a predefined sequence.

Example:

NORMAL JOURNEY

0 sec:

Device online

5 sec:

Temperature increases

10 sec:

Temperature crosses threshold

15 sec:

Alert generated

20 sec:

Network disconnects

25 sec:

Offline records accumulate

35 sec:

Network reconnects

40 sec:

Data synchronized

45 sec:

Blockchain verification

50 sec:

Journey complete

This will make the project extremely effective during a live presentation.

=====

38. DEMO STORY

=====

Create a default shipment:

MANGO-001

Product:

Alphonso Mango

Origin:

Erode, Tamil Nadu

Destination:

Dubai, UAE

Sensor:

AG-SENSOR-001

Start shipment.

Show normal conditions.

Then trigger:

TEMPERATURE SPIKE

The dashboard should show:

WARNING

Then:

NETWORK OUTAGE

Show:

OFFLINE MODE

Sensor data continues.

Then:

NETWORK RECOVERY

Show:

24 RECORDS SYNCHRONIZED

Then:

BLOCKCHAIN VERIFIED

Then:

SIMULATE DATA TAMPERING

Show:

DATA INTEGRITY FAILURE

Then restore the correct data.

Show:

DATA INTEGRITY VERIFIED

This should be possible entirely from the UI.

=====

39. RESPONSIVE DESIGN

=====

The entire application must work on:

Desktop

Laptop

Tablet

Mobile

Dashboard must adapt.

Tables must become horizontally scrollable or card-based.

Sidebar becomes mobile drawer.

Charts resize correctly.

QR verification must work well on mobile.

=====

40. ACCESSIBILITY

=====

Use:

Semantic HTML

Keyboard navigation

Readable contrast

Accessible labels

ARIA where required

Focus states

Do not rely only on colors to communicate status.

Use:

✓ Verified

⚠ Warning

✗ Critical

along with colors.

=====

41. SECURITY

=====

Implement:

Password hashing

JWT authentication

Role-based authorization

Input validation

API validation

Rate limiting where appropriate

Secure environment variables

No hard-coded secrets

Never expose:

Private keys

Fabric credentials

Database credentials

JWT secrets

in frontend code.

Use:

.env

and provide:

.env.example

=====

42. PROJECT STRUCTURE

=====

Use a clean architecture.

Example:

/frontend

 /src

 /components

 /pages

 /layouts

 /hooks

 /services

 /contexts

 /utils

 /types

/backend

 /src

 /controllers

 /routes

 /services

 /models

 /middleware

 /simulation

 /blockchain

 /mqtt

/utils

/blockchain

/chaincode

/network

/scripts

/docs

README.md

=====

43. COMPONENT REUSABILITY

=====

Create reusable components:

KpiCard

StatusBadge

SensorCard

SensorChart

ShipmentCard

ShipmentTable

AlertCard

AlertTable

Timeline

BlockchainTransaction

VerificationBadge

QRCodeCard

Map

DeviceCard

Modal

Toast

DataTable

EmptyState

LoadingSkeleton

=====

44. MAP

=====

Use Leaflet or another suitable map library.

Show shipment routes.

Example:

Erode



Coimbatore



Chennai



Dubai

Animate the shipment marker gradually.

Do not require paid API keys if possible.

Use OpenStreetMap-compatible mapping.

=====

45. VISUAL DETAILS

=====

Use professional typography.

Suggested:

Inter

Manrope

or another clean modern sans-serif.

Use:

8px spacing system.

Cards:

12–16px radius.

Buttons:

6–10px radius.

Animations:

150–300ms.

Use subtle transitions.

Do not animate everything.

=====

46. ERROR HANDLING

=====

Every API request must have:

Loading

Success

Error

Example:

Loading:

"Loading shipment data..."

Error:

"We couldn't load this shipment."

Retry button.

=====

47. EMPTY STATES

=====

Never show blank pages.

Example:

"No active shipments"

"Create your first shipment to begin tracking your supply chain."

Button:

"Create Shipment"

=====

48. SEARCH

=====

Global search should find:

Shipment ID

Device ID

Product

Origin

Destination

Transaction ID

=====

49. DATA EXPORT

=====

Allow CSV export for:

Telemetry

Shipments

Alerts

Blockchain transactions

=====

50. FOOTER

=====

Footer:

AGRICHAIN

"Trust Every Journey."

Links:

About

Technology

Privacy

Terms

Contact

Add:

© 2026 AgriChain

=====

51. IMPORTANT ENGINEERING REQUIREMENTS

=====

Do NOT create a fake static website.

The following must actually work:

Authentication

Navigation

CRUD shipments

IoT simulation
Real-time sensor updates
Alerts
Offline mode
Synchronization
Hash generation
Integrity verification
Blockchain abstraction
Chain-of-custody
QR generation
QR verification
Analytics
Search
Filtering
Demo scenarios

Use real application state.

Avoid hard-coded UI values wherever possible.

=====

52. FUTURE MQTT ARCHITECTURE

=====

Even though physical IoT is not implemented now, prepare the architecture for:

ESP32

↓

MQTT

↓

MQTT Broker



Backend MQTT Subscriber



Telemetry Processing



Database



Blockchain



WebSocket



Dashboard

Create an MQTT service interface.

Example:

```
mqttService.subscribeToDevice(deviceId)
```

```
mqttService.publishTelemetry(deviceId, data)
```

```
mqttService.disconnect()
```

The simulator should implement the same telemetry interface.

This will allow future replacement:

IoT Simulator



TelemetryInterface



|
RealMQTTService

=====

53. FUTURE ENERGY HARVESTING SUPPORT

=====

Do not implement physical energy harvesting now.

But the device model should support:

energySource:

"SOLAR"

"THERMAL"

"BATTERY"

"HYBRID"

Show:

Energy source

Battery percentage

Estimated health

This prepares the application for future hardware.

=====

54. DOCUMENTATION

=====

Create a comprehensive README.

Include:

Project overview

Architecture

Technology stack

Installation

Environment variables

Frontend setup

Backend setup

Database setup

Blockchain setup

Simulation setup

Demo accounts

Demo scenarios

API documentation

Future MQTT integration

Future hardware integration

Also create:

ARCHITECTURE.md

Explain:

IoT layer

MQTT

Backend

Database

Blockchain

Frontend

Offline synchronization

Security

Data integrity

=====

55. FINAL ACCEPTANCE CRITERIA

=====

The project is complete ONLY when:

- ☐ Landing page works
- ☐ Login works
- ☐ Role-based dashboard works
- ☐ Shipment creation works
- ☐ IoT simulator works
- ☐ Real-time readings update
- ☐ Charts update
- ☐ Alerts trigger automatically
- ☐ Offline mode works
- ☐ Local pending data works
- ☐ Synchronization works
- ☐ SHA-256 hashing works
- ☐ Tampering simulation works
- ☐ Integrity failure is detected
- ☐ Blockchain abstraction works
- ☐ Hyperledger adapter exists
- ☐ Mock blockchain adapter works when Fabric unavailable
- ☐ Blockchain explorer works
- ☐ Chain of custody works
- ☐ QR generation works
- ☐ QR verification works

- ☐ Digital passport works
- ☐ Analytics works
- ☐ Reports work
- ☐ Search works
- ☐ Filters work
- ☐ Demo mode works
- ☐ Responsive design works
- ☐ Error handling works
- ☐ README exists
- ☐ .env.example exists
- ☐ No secrets committed

=====

56. MOST IMPORTANT PRODUCT PRINCIPLE

=====

The platform must communicate one simple idea:

"Even when the network disappears, the trust does not."

The IoT layer provides the physical-world data.

The local storage provides resilience.

Cryptographic hashing provides data integrity.

Hyperledger provides tamper-evident shared traceability.

The web application makes the entire system understandable and usable.

Do not build isolated features.

Everything must connect into ONE coherent user journey:

CREATE SHIPMENT



ASSIGN SENSOR



MONITOR CONDITIONS



DETECT EVENT



STORE DATA



NETWORK OUTAGE



CONTINUE LOCAL LOGGING



NETWORK RECOVERY



SYNC DATA



HASH DATA



BLOCKCHAIN RECORD



VERIFY INTEGRITY



UPDATE CHAIN OF CUSTODY



GENERATE DIGITAL PASSPORT



QR VERIFICATION

=====

57. DEVELOPMENT PRIORITY

=====

Build in this order:

PHASE 1:

Project setup + design system

PHASE 2:

Landing page + authentication

PHASE 3:

Application shell + dashboard

PHASE 4:

Shipment management

PHASE 5:

IoT simulation engine

PHASE 6:

Live monitoring

PHASE 7:

Alerts

PHASE 8:

Offline mode + synchronization

PHASE 9:

Cryptographic integrity

PHASE 10:

Blockchain abstraction + Fabric adapter

PHASE 11:

Blockchain explorer

PHASE 12:

Chain of custody

PHASE 13:

Digital passport + QR

PHASE 14:

Analytics + reports

PHASE 15:

Demo mode

PHASE 16:

Testing + polishing

DO NOT skip directly to visual polish before core functionality works.

=====

58. FINAL UI QUALITY REQUIREMENT

=====

Before considering the project complete, review every page as if it were being released by a real company.

Ask:

Does this look trustworthy?

Can a non-technical farmer understand it?

Can an exporter quickly identify a problem?

Can an auditor verify the history?

Can an importer verify the shipment?

Does the application clearly communicate what happened?

Does every action have useful feedback?

Does the interface look intentionally designed?

If any answer is NO, improve the UI/UX.

=====

FINAL INSTRUCTION:

Build the complete working application.

Do not merely generate mock screens.

Implement the actual frontend, backend, simulation engine, database integration, cryptographic hashing, blockchain abstraction, QR workflow, authentication, role-based access, alerts, synchronization, and demo scenarios.

When a real Hyperledger Fabric network is unavailable, use the MockBlockchainProvider without breaking the application, while keeping the Hyperledger Fabric provider ready for actual integration.

The final result should feel like a startup-quality enterprise agricultural traceability platform that could be demonstrated to:

- Hackathon judges
- Agricultural exporters
- SMEs
- Logistics companies
- Food safety auditors
- Government/industry stakeholders

The final demo should tell a clear story:

A shipment leaves a farm.

Its environment is continuously monitored.

The network goes down.

The device continues recording.

The network returns.

Data synchronizes.

The data is cryptographically verified.

The blockchain records the evidence.

A tampering attempt is detected.

The shipment receives a trusted digital passport.

Anyone authorized can verify its journey.

PRODUCT TAGLINE:

"AGRICHAIN — Trust Every Journey."

=====